HAUTE ÉCOLE D'INGÉNIERIE ET DE GESTION DU CANTON DE VAUD

Bachelor Thesis

Modernisation and reliability improvement of the ICNML biometric trace platform

Student	**Quentin Surdez**
Supervisor	Prof. Sylvain Pasini
Department	Technologies de l'information et de la communication (TIC)
Faculty	Informatique et systèmes de communication (ISC)
Orientation	Logiciel (ISC-L)
Mandating Company	UNIL - ESC Christophe Champod Avenue François-Alphonse Forel 1015 Lausanne

Academic year	2025-26

Yverdon-les-Bains, 24.03.2026

Authentication

I hereby certify that I have completed this work myself and have used no other sources than those expressly mentioned.

Quentin Surdez

Yverdon-les-Bains, 24.03.2026

Preamble

This Bachelor Thesis (hereinafter BT) is completed at the end of the study program, with a view to obtaining the title of Bachelor of Science HES-SO in Engineering.

As an academic work, its content, without prejudging its value, does not engage the responsibility of the author, nor that of the Bachelor Thesis jury and the School.

Any use, even partial, of this BT must be made in compliance with copyright law.

HEIG-VD
The Head of Department TIC

Yverdon-les-Bains, 24.03.2026

Abstract

Bachelor Thesis 2025-26
Title: Modernisation and reliability improvement of the ICNML biometric trace platform
Subtitle:

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut.

Student	**Quentin Surdez**
Supervisor	Prof. Sylvain Pasini
Mandating Company	UNIL - ESC

Specifications

Problem summary

ICNML (*International Close Non-Matches Library*) is an open-souce platform designed for handling biometric traces in scientific and experimentation contexts. It is deployed by several research institutions, including the University of Lausanne (*UNIL*). It is used by collaborators worldwide on projects focused on image analysis and biometric recognition.

Problem statement

The system was initially developed in a research environment that prioritized functionnality and flexibility over long-term maintenance and technical standardisation. Today, the plateform remains a central component in many scientific activities. However, its evolution and sustainability are limited by accumulated technical debt. These weaknesses affect maintenability, security, and usability for both admins and end users alike.

Specifications

Objectives

This aim of this bachelor thesis is to strenghten the security and maintenability of the **ICNML** platform. The work is built around the 3 following objectives.
The first objetive is to **reinforce technical maintainability** by producing documentation of the system architecture and all critical processes. The underlying objective is to enable future contributors to understand, install, and develop the platform easily compared to now.
The second objective is to **improve security** by bringin transparency to internal processes:

- data encryption in the db
- access control mechanisms
- watermark on the images downloaded
- encryption mechanisms applied per donor

The third objective is to **demistify and reinforce management operations** such as installation and backup restoration. This will improve maintanability and reusability by new developers.

The third objective is to **optimise user experience** through filtering and management features. Especially filtering by donor or image quality.

Schedule

This work begins the 16.02.2026 and ends the 23.07.2026. Over the 16 first weeks, from 16.02 to 08.06, the workload is 12 hours per week. The 6 last weeks, from 08.06 to 27.07, the workload is the equivalent of a full-time.
An assessed intermediary submission is due on 20.05. The final report is due on the 27.07 at 12h00.
The defense will be organized between the 24.09 and the 11.10.

Tasks

The work is built into seven phases, each one building ontop of the previous one.
The **Starting phase** (16-19.02) covers the creation of an initial planning document and the drafting of this specification.
The **First steps in the codebase phase** (24.02-02.04) involves an in-depth analysis of the current codebase, starting with a first look around the application, followed by the setup of a reproducible local development environment. It then covers the documentation of the system schema and development environment, and closes with the creation of a documentation template.
The **User management phase** (07.04-16.04) analyses how the keys for the users are generated and stored, how their biometric data are encrypted and how the authentication is managed within the application.
The **Backup management phase** (07.05-12.05) analyses the process behind the backup and restoration mechanism.
The **Deployment management phase** (12.05-20.05) analyses the current deployment solution and produces corresponding documentation. By the end of this phase, the created documentation will give enough information to take a decision on which functional improvements will be implemented during the development phase.
The **Development phase** (20.05-10.07) covers the implementation of the improvements selected. The exact scope is to be determined based on the findings of the precedent phases.
The **Admin end of the project phase** (13.07-23.07) will focus on writing the final report and the different admin tasks that comes with the end of this bachelor thesis such as a publishable summary and a poster.

Deliverables

Here are the expected deliverables :
1. Technical documentation on:
   - System architecture
   - Per-donor image encryption process with key generation and storage

- Digital watermarking process applied to downloaded images
- Backup mechanism
- Deployment procedure

2. Functional improvements:
   - Scope defined on 20.05, once documentation is complete
   - Candidate features are as followed:
     - Sorting by donor
     - Sorting by image quality
3. End of thesis deliverables:
   - An intermediary report due on 20.05
   - A final report due on 23.07
   - A publishable summary and a poster

Table of contents

Authentication **3**

Preamble **4**

Abstract **5**

Specifications **6**
Problem summary 6
Specifications 6

1 Introduction **12**

2 Planning **13**
2.1 Initial planning 13

3 ICNML: context **16**
3.1 What is ICNML ? 16
3.2 Scientific context and motivation 16
3.3 Data acquisition to exercises creation 16

4 Repository structure **18**
4.1 List of directories 18
4.2 Description 18
4.3 Schema 20

5 Roles and permissions **22**
5.1 Account types 22
5.2 Authentication and Session Model 23
5.3 Access-Control decorators 23

5.4 Role descriptions ... 24
5.5 Trainer (type 4) ... 25

6 DEK Donor Generation ... 28
6.1 Prerequisites and Roles ... 28
6.2 Steps ... 28

7 Database Architecture ... 32
7.1 `account_type` table ... 32
7.2 `users` table ... 32
7.3 `webauthn` table ... 33
7.4 `cf` table ... 34
7.5 `donor_dek` table ... 34
7.6 `signin_requests` table ... 35
7.7 `submissions` table ... 36
7.8 `files_type` table ... 36
7.9 `files` table ... 37
7.10 `files_v` view ... 38
7.11 `segments_locations` table ... 38
7.12 `files_segments` table ... 39
7.13 `files_segements_v` view ... 39
7.14 `thumbnails` table ... 39
7.15 `quality_type` table ... 40
7.16 `tenprint_zones_location` table ... 40
7.17 `tenprint_zones` table ... 41
7.18 `mark_info` table ... 42
7.19 `gp` table ... 42
7.20 `donor_fingers_gp` table ... 43
7.21 `pc` table ... 44
7.22 `detection_technics` table ... 44
7.23 `surfaces` table ... 45
7.24 `exercises` table ... 46
7.25 `exercises_folder` table ... 46
7.26 `exercises_trainee_list` table ... 47
7.27 `activities` table ... 47
7.28 `distortion` table ... 48

8 Architecture ... 50

9 Implementation ... 51

10 Results **52**

11 Conclusion **53**

Bibliography **54**

Appendices **59**

I Tools used **60**

II Work diary **61**

1 Introduction

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine metu degendae praesidia firmissima. – Filium morte multavit. – Si sine causa, nollem me ab eo delectari, quod ista Platonis, Aristoteli, Theophrasti orationis ornamenta neglexerit. Nam illud quidem physici, credere aliquid esse minimum, quod profecto numquam putavisset, si a Polyaeno, familiari suo, geometrica discere maluisset quam illum etiam ipsum dedocere. Sol Democrito magnus videtur, quippe homini erudito in geometriaque perfecto, huic pedalis fortasse; tantum enim esse omnino in nostris poetis aut inertissimae segnitiae est aut fastidii delicatissimi. Mihi quidem videtur, inermis ac nudus est. Tollit definitiones, nihil de dividendo ac partiendo docet, non quo ignorare vos arbitrer, sed ut.

2 Planning

2.1 Initial planning

This is the initial planning for the bachelor thesis. It explains the different tasks needed to complete the project. The tasks have their estimated hours needed to complete next to them in parentheses (x). Some tasks have greater estimated hours than their complete subtasks, it is because the hours estimated to write the reports are within the total estimate. \
It is important to know the work begins the 16.02.2026 and ends the 23.07.2026. Over the 16 first weeks, from 16.02 to 08.06, the workload is 12 hours per week. The 6 last weeks, from 08.06 to 27.07, the workload is the equivalent of a full-time.

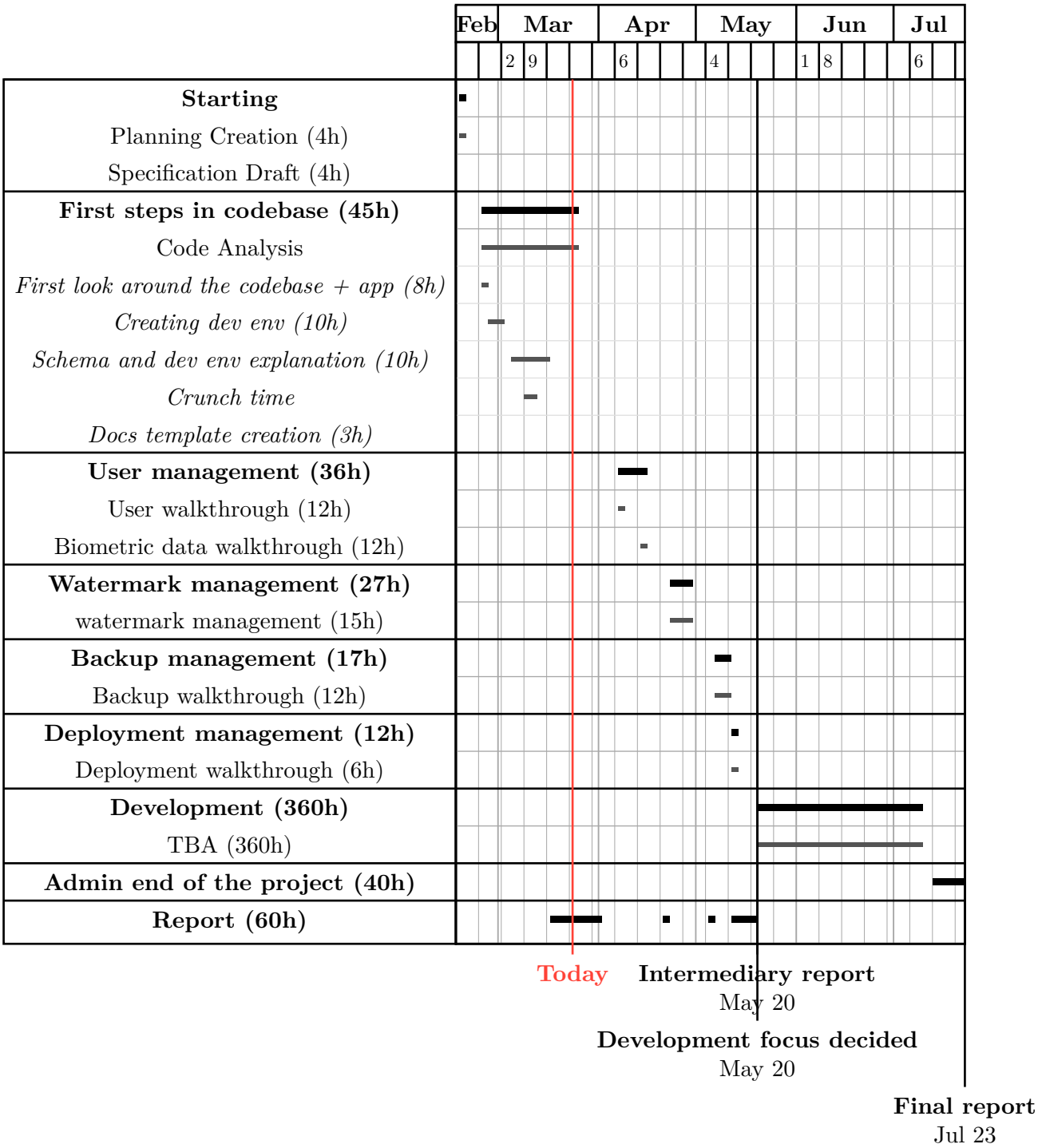
Feb
Mar
Apr
May
Jun
Jul
2
9
6
4
1
8
6
Starting
Planning Creation (4h)
Specification Draft (4h)
First steps in codebase (45h)
Code Analysis
First look around the codebase + app (8h)
Creating dev env (10h)
Schema and dev env explanation (10h)
Crunch time
Docs template creation (3h)
User management (36h)
User walkthrough (12h)
Biometric data walkthrough (12h)
Watermark management (27h)
watermark management (15h)
Backup management (17h)
Backup walkthrough (12h)
Deployment management (12h)
Deployment walkthrough (6h)
Development (360h)
TBA (360h)
Admin end of the project (40h)
Report (60h)
Today
Intermediary report
May 20
Development focus decided
May 20
Final report
Jul 23

Figure 1: Inital planning for the project

3 ICNML: context

3.1 What is ICNML ?

ICNML (International Close Non-Matches Library) is a web platform designed for managing biometric data in scientific and forensic research contexts. It is used by several reasearch institutions, for example at the University of Lausanne, Ecole des Sciences Criminelles (UNIL-ESC). It supports research in fingerprint identification.
The platform is both a library and a work environment for fingerprint traces (marks) and reference prints (tenprints), supporting all the steps from data acquisition to creating exercises for examiner training.

3.2 Scientific context and motivation

Fingerprint examination in forensic science relies on comparing a latent mark found at a crime scene with a reference database. The Automated Fingerprint Identification Systems (AFIS) return a ranked list of candidate matches. The candidates that rank highly but do not actually correspond to the source trace are called Close Non-Matches (CNM). These are particularly valuable in research as they represent challenges for identification.
ICNML was built to collect, organise, and exploit these challenges for identification. It has two missions:

1. Acts as a recipient for donors' biometric data (traces and reference tenprints) and for close non-matches results returned by AFIS searches.
2. Provide trainers with the annotated data they need to build exercises that test fingerprint examiners against difficult, realistic scenarios.

3.3 Data acquisition to exercises creation

There are three different steps to understand the ICNML workflow from data creation to exercises creation.

3.3.1 Stage 1: data acquisition

A Submitter registers a new Donor in the system, for more info on the roles see Section 5. The donor provides fingerprint reference cards (tenprints) and latent traces (marks). Consent forms are acquired at this stage for new donors. Upon registration, a per-donor cryptographic key (DEK, see Section 6) is generated and all biometric data for that donor is stored encrypted.

3.3.2 Stage 2: AFIS search

An Administrator creates an AFIS target by submitting a trace to the AFIS users. The AFIS users will submit the target created by the Administrator to an Automated Fingerprint Identification System. The AFIS returns a ranked list of candidate matches. The AFIS user will annotate those candidates, records a PFSP decision for each, a quality score that characterises the quality of the close non-match.
This will grow the library of close non-matches on ICNML.

3.3.3 Stage 3: Trainer exercise creation

A Trainer uses the mark data, together with the known true correspondence and the AFIS close non-matches to build training exercises. For each exercise the trainer can choose to include either correct matches or misleading close non-matches.
This allows the trainer to create realistic examination scenarios. Then the trainer can download the exercises they created and give them to their team.

4 Repository structure

This chapter focus on the repositories structure as well as their interaction with one another. It is used as a backbone to understand the dependencies of the directories as well as how they're linked together through the CICD.

4.1 List of directories

- afis_assignments
- base
- cdn
- docker
- documents
- fingerprintexperts_assessment
- config/production
- tools
- web

4.2 Description

4.2.1 Repo afis_assignemnts

This repository contains one R file. This file is name `TrialSortingScript` and relies on a csv file that is not present. This seems to be filtering out some data from labs. This seems to be outside the scope of this thesis. It may be removed in the future.

4.2.2 Repo base

This repository contains on Dockerfile as well as a gitlab cicd file and a license. The Dockerfile is a minimal debian install. It updates, upgrades and installs `gettext-base`. It then removes the `/var/lib/apt/lists/*`.
The cicd, when run, uses Kaniko to build the Docker image and pushes it to Gitlab's container registry with two tags. The commit SHA and latest. It has only seen 7 commits and the last one dates from 2022.

There are two images in the container repository of this repo:
- base/cache
- base

4.2.3 Repo cdn

This repo hosts all the js and css that is considered as libraries for the web application. Here's an exhaustive lists of what is present within the repo:
- chosen
- aes
- pbkdf2
- dropzone
- jquery
- loadingcss
- md5.min
- rangeslider.min
- toastr.min
- base64
- moment.min
- otplib-browser
- sha512
- underscore-min

4.2.4 Repo docker

This repo has the description "ICNML application builder process". From the README, we can understand that the docker repo has submodules. After testing, today it cannot reach them. This repository is mainly focused on the CICD pipeline.
From the timestamps of the last commit on the web repository and this one, it seems there's an automation in place to keep this repository updated with the web one.
The README emphasizes that this is not for development and then gives info on how to setup a development environment with the submodules. It is not clear what the responsability or scope of this repository is.
We can see that the web repository is a submodule of this one.
There are 3 images in the container repository of this repo:
- docker/web/cache
- docker/redis
- docker/web

4.2.5 Repo documents

There is only one pdf file within this repository. It is a donor consent form.

4.2.6 Repo fingerprintexperts_assessment

This repo contains the results from tests on ICNML images with fingerprint experts. It's out of scope. It may be deleted later on.

4.2.7 Repo config/production

This repo seems to be a part of an automated process. The timestamps of the last commits correspon to the ones on the web repository.

4.2.8 Repo scripts

This repo hosts scripts used database management. There is one for creating the backup, one for cleaning the backups, one to duplicate the database and one for combining gpg keys.

4.2.9 Repo tools

This repo contains scripts in python to migrate data and convert it. It relies on submodules that are not accesible today.

4.2.10 Repo web

This repo is where the web application code is hosted. It has the cdn repo as submodule, however I also see some dependencies on MDmisc and NIST libraries. There is a flask application, sql schema for a database. There are also a lot of utils files related to encryption, so I think this is where I will spend most of my time after creating a dev environment.

4.3 Schema

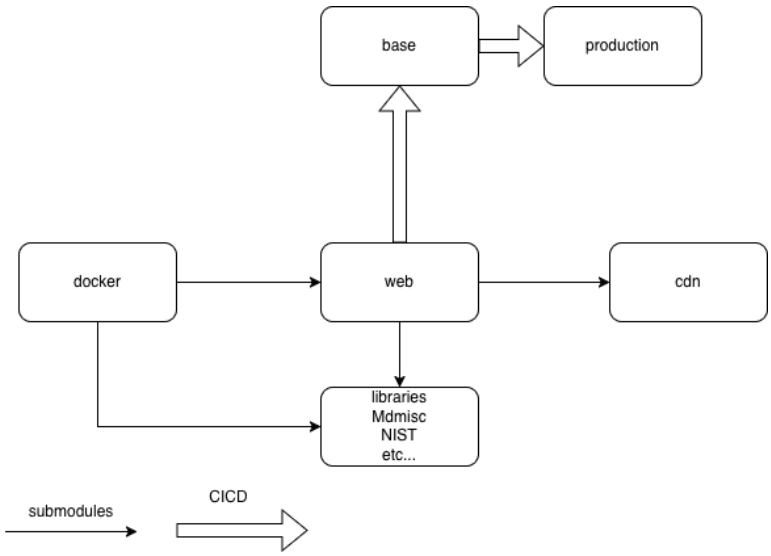

Figure 2: Repositories dependencies

5 Roles and permissions

The application ICNML follows a role-based access control (RBAC) model. Every authenticated user has one and only one role. The different roles possible within the application are stored in the `account_type` sql table. It's reflected on login in the Redis session object. A series of decorator are used to manage the authorisation of each route.
This chapter describes each role: how they are created, what routes and database operations are allowed.

5.1 Account types

There are 6 account types in the `01-account_type.sql` file. A flag `can_singin` exists for each account type. There is a type and it shoud be called `can_signin`. It filters the types for which a user can request a user account via the `GET /signin` public form.

ID	Name	Can request account	Short description
1	Administrator	No	Full privileged access; manages the platform.
2	Donor	No	Subject whose biometric data is collected, created by a Submitter.
3	Submitter	Yes	Registers donors.
4	Trainer	Yes	Uses mark data to train fingerprint examiners.
5	AFIS	Yes	Works with Automated Fingerprint Identification System targets and candidate matches.
6	Selection	Yes	Selection user, does not seem to have specific logic.

Table 1: Account types table

5.2 Authentication and Session Model

The authentication is handled by the `/login` route. It expects the user to complete a form and then it will recall the `POST /do/login` endpoint for each attribute (username, password, TOTP or WebAuthn passkey). Then, these are the keys that are written in Redis.

```python
session[ "account_type" ]
session[ "account_type_name" ]
session[ "logged" ]
session[ "username" ]
session[ "user_id" ]
```

Listing 1: Keys written in Redis' session

5.3 Access-Control decorators

Most of the authorisation logic lies in the decorators file. This is done via the use of `functools` package to make decorator out of functions.

5.3.1 `@login_required`

The first decorator is the one that requires the user to be logged in. It checks that the `session["logged"]` key is set to `True`. Any unauthenticated.

5.3.2 `@admin_required`

This decorator checks wether the `account_type_name` key in the session is set to Administrator. It also checks wether or not the user is logged in with the same logic as the `login_required` decorator.
If the the account type name is not Administrator, it will redirect to login.

5.3.3 `@submission_has_access`

This decorator will check wether the user has access to the current submission.

- If the user is an Administrator, then they can access the submission
- If the user is a Submitter, the submission needs to be theirs (this needs clarification, I don't understand it yet)
- All other roles are redirected to the login page

5.3.4 `@trainer_has_access`

This decorator checks wether the user has access to the trainer specific endpoints.

- If the user is an Administrator, then they can access it
- If the user is a Trainer and is logged then they can access it

- All other roles are redirected to the login page.

5.4 Role descriptions

5.4.1 Administrator (type 1)

Creation: Accounts must be created directly in the database or by a process not documented as of yet.
Home page redirect: `GET /admin/submission/list`
Scope: The Administrator has unrestricted access to the entire application. Every decorator allows Administrators. Dedicated `/admin/*` routes are only usable by them.
Capabilities:
- **New user management**: can validate pending sign in requests via `GET /validate_signin`
- **Submission overview**: can view all submissions via `GET /admin/submission/list`
- **AFIS management**: creates targets (`POST /admin/target/<submission_id>/<pc>/new`), deletes targets or candidate matches, updates assigned AFIS users, and batch-assigns targets (`POST /admin/afis/batch_assign/do`)
- **Full-resolution images:** downloads uncompressed originals via `GET /image/file/<id>/full_resolution`.
- **PiAnoS integration:** synchronises all users and segments to the external PiAnoS system via `GET /pianos_api/add_user/all` and `GET /pianos_api/add_segments/all`.
- **Mark deletion (admin context):** `POST /admin/<submission_id>/mark/<m_id>/delete`.
- **UUID inspection:** retrieves the raw database table for any UUID via `GET /uuid/get_table/<uuid>`.

5.4.2 Donor (type 2)

Creation: Created by a Submitter via `POST /submission/do_new`. No self-registration.
Home page redirect: `GET /user/myprofile/dek`
Scope: Donors manage only their own cryptographic key (DEK) and view their own biometric data. They cannot interact with the submission or AFIS workflows.
Permitted operations:

Route	Purpose
`GET /user/myprofile/dek`	View DEK profile page.
`GET /user/myprofile/tenprint`	View own tenprint cards.
`GET /user/myprofile/marks`	View own mark images.
`POST /dek/reconstruct`	Re-derive the DEK from the donor's e-mail (requires re-authentication).
`GET /dek/delete`	Soft-delete the DEK (`donor_dek.dek` set to `NULL`; reversible).

Route	Purpose
`GET /dek/fulldelete`	Permanently delete the DEK row (right-to-erasure; irreversible).

Table 2: Routes of interest for Donor role

5.4.3 Submitter (type 3)

Creation: Self-registration via `GET /signin` / `POST /do/signin` followed by e-mail confirmation and administrator approval.
Home page redirect: `GET /submission/list`
Scope: Submitters manage the complete lifecycle of their own submissions.
Permitted operations (own submissions only):

Action	Routes
Create donor + submission	`POST /submission/do_new` — creates `users`, `donor_dek`, and `submissions` rows.
Upload files	`POST /upload`, `/submission/<id>/add_files`, `/submission/<id>/add_marks`, `/submission/<id>/consent_form`.
Annotate tenprints	Set template, quality, segment coordinates, and general pattern on tenprint cards.
Annotate marks	Set PFSP?, and delete marks.
Manage submission	Set nickname, set GP, delete submission, view targets.
Browse own data	`GET /submission/list`, tenprint list, mark list, segment views.

Table 3: Routes of interest for Submitters role

Ownership enforcement: `@submission_has_access` issues HTTP 403 if the `submission_id` in the URL was not created by the current submitter.

5.5 Trainer (type 4)

Creation: Self-registration via `GET /signin` (Needs a confirmation from admin).
Home page redirect: `GET /marks/search`
Scope: Trainers consume mark and tenprint data for examiner-training exercises. Their access is read-oriented; they do not create submissions or manage donors.
Permitted operations:

Route	Purpose
`GET /marks/search`	Search and filter marks for training.
`GET /marks/exercise/<id>`	View a training exercise.

Route	Purpose
`GET /marks/folder/<id>`	Browse an exercise folder.
`POST /exercises/add_tenprint`	Associate a tenprint card with an exercise.

Table 4: Routes of interest for Trainer role

5.5.1 AFIS (type 5)

Creation: Self-registration via `GET /signin` (Needs a confirmation from admin).
Home page redirect: `GET /afis/list/targets`
Scope: AFIS users work on fingerprint identification targets: they receive candidate match assignments, upload and annotate search results, and set comparison decisions. This seems like a big part of the application.
Permitted operations:

Route	Purpose
`GET /afis/list/targets`	List assigned AFIS targets.
`GET /afis/incidental/donors/list`	List incidental donors.
`GET /afis/incidental/donor/<uuid>/list`	Browse a specific incidental donor's data.
`GET /afis/<uuid>`	View an AFIS target.
`GET /afis/<uuid>/download{,/mark,_exercise}`	Download target data, mark, or exercise package.
`GET /afis/<uuid>/upload/list`	List uploaded result files.
`GET /afis/<uuid>/upload/new/<type>`	Initiate a result file upload.
`GET /afis/<target>/<cnm>`	View a candidate match.
`POST /afis/<target>/<cnm>/set_pfsp`	Record the PFSP decision for a candidate match.
`POST /afis/<target>/<cnm>/upload`	Upload a candidate match result file.
`POST /afis/<target>/<cnm>/update_field`	Update a field on a candidate match.
`GET /afis/<cnm>/<file>/<fpc>/autodetect`	Auto-detect minutiae from a mark file.
`GET /afis/<cnm>/<file>/autodetect/tiff`	Auto-detect from a TIFF file.
`GET /afis/<cnm>/<file>/<fpc>/res`	Retrieve image resolution metadata.

Route	Purpose
`GET /image/cnm_candidate/screenshot/<file>/preview`	Preview a candidate screenshot.

Table 5: Routes of interest for AFIS role

5.5.2 Selection (type 6)

Creation: Self-registration via `GET /signin` (Needs a confirmation from admin).
Home page redirect: Default (`/`) — no specialised redirect is defined in `views/base/__init__.py` for this role.
Scope: The Selection role is defined in the database and the registration flow but has no dedicated routes or decorators at this time. A Selection user can log in and reach any route guarded only by `@login_required` (shared utilities, UUID search, image preview, donor views, etc.), but cannot access submission, AFIS, trainer, or admin-specific routes.

6 DEK Donor Generation

When a new donor is registered within the ICNML application, the first cryptographic operation is the generation of a **Data Encryption Key** (DEK). This key is unique to the donor and is used to encrypt all biometric data associated with them.
The DEK is designed to be completely **uncoupled** with the rest of the application so that deleting it will render all biometric data within the database unrecoverable. This is a very important workflow to guarantee a kind of privacy-by-deletion design. If the user wants to delete their key, they have the possibility to do it, and it will render all their biometric data unreadable by other users.
This document describes the complete DEK generation workflow and is used as a stepping stone for future documentation. This will cover the HTTP routes used, the cryptographic steps, and the object stored in the database to fully understand the process.

6.1 Prerequisites and Roles

The donor creation process, by derivation the DEK generation, requires a **Submitter** and the request from the user to be a **Donor**.

6.2 Steps

6.2.1 Entry point

The entry point for the generation of the donor dek is `POST /submission/do_new`. This is triggered after the **Submitter** has sent the form to register the new **Donor**. This is only accessible for the users with **Administrator** or **Submitter** role.
The form provides two inputs:

- `email`, this is the donor's plain-text email
- `upload_nickname`, this is the submission nickname

6.2.2 Step 1, Duplicate Check

Before any creation, the code checks wether the email is already used within the db. To be more precise from the sql query, it will check if, among the submissions done by the **Submitter**, one has the same email hash.

```python
sql = "SELECT id, email_hash FROM submissions WHERE submitter_id = %s"
...
if utils.hash.pbkdf2( email ).verify( case[ "email_hash" ] ):
# Return error that email is used for another submission
```

Listing 2: Code for duplicate check

We can see here that the provided email is hashed and then is verified against the `email_hash` stored in the database. If a match is found the request is rejected with an error.

6.2.3 Step 2, Donor User Creation Account

In this step, the **Donor** is created with the status *pending*. The `email` and the `upload_nickname` are both encrypted using the aes functions found in `utils/aes.py`. Then an id is retrieved via a sequence found in the database.
A new user is inserted within the table `users` with the format `donor_<id>`. Here is stored the username, the email as well as the type.

```python
    userid = config.db.query_fetchone( "SELECT nextval( 'username_donor_seq' ) as id" )[ "id" ]
    username = "donor_{}".format( userid )
    sql = utils.sql.sql_insert_generate( "users", [ "username", "email", "type" ], "id" )
```

Listing 3: Creation of a new donor

6.2.4 Step 3, DEK Generation

After creating a user of type **Donor**, the DEK is generated calling the `utils.encryption.dek_generate()` function. This function can take up to 4 arguments:

- `email`, the email of the donor
- `email_hash`, the hash of the donor's email
  - Either `email` or `email_hash` must be present in the kwargs
- `username`, the username of the donor (required)
- `salt`, the salt used for key generation (optional)

The function will hash the username and the email hash given using pbkdf2 to create the dek.

```python
dek = hash.pbkdf2(
    "{}:{}".format( username, email, ),
    dek_salt,
    iterations = config.DEK_NB_ITERATIONS,
    hash_name = "sha512"
).hash( True )
```

Listing 4: Code for the DEK generation

The `config.DEK_NB_ITERATIONS` is 500′000.
The function creates a check object with a call to `aes.do_encrypt()` with a json as parameters and the dek. It is used later on if the user chooses to recreate their DEK after a soft-delete.
The function returns 3 variables:

- `dek_salt`, the salt used for generating the DEK
- `dek`, the generated DEK
- `check`, the encrypted DEK for rebuilding the DEK later on to check against

6.2.5 Step 4, DEK persistance

After the creation of the DEK, it is stored within the `donor_dek` table. Here's a list of the different fields that are persisted:

- `donor_name`, this is the username created previously `donor_<id>`
- `salt`, this is the `dek_salt` returned by the dek generating function
- `dek`, the generated DEK
- `dek_check`, the encrypted DEK using AES
- `iterations`, the number of iterations used to create the DEK
- `algo`, the algo used for generating the DEK, here always pbkdf2
- `hash`, the structure of the hash used, here always sha512

It then returns a response with error false and the donor uuid.

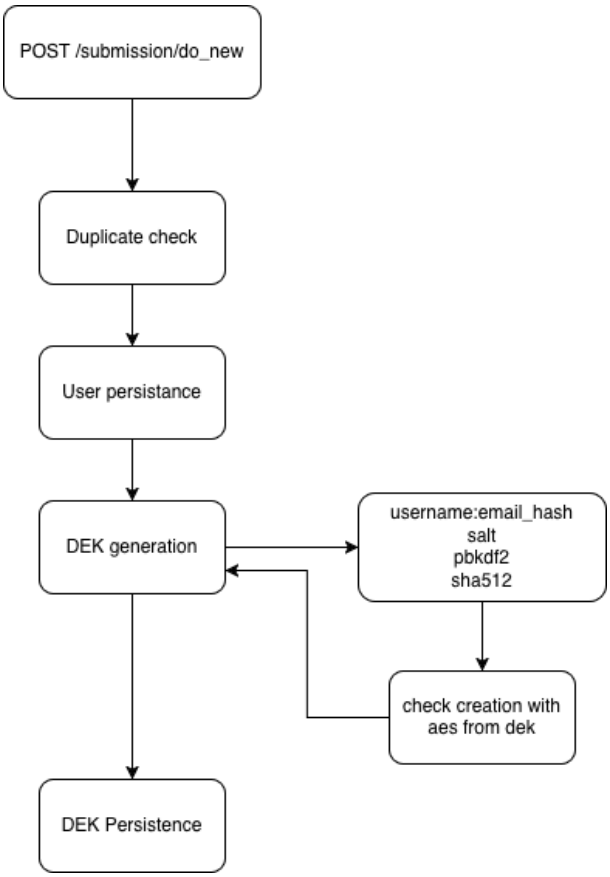

Figure 3: DEK Generation Schema

7 Database Architecture

The *ICNML* application uses a PostgreSQL database named `icnml` owned by the `icnml` role. All the tables, sequences and views live in the public schema. This chapter documents each object in the databse with details.

7.1 `account_type` table

Stores the different type of account in the web application.

Column	Type	Nullable	Notes
`id`	`integer`	No	Primary key, auto-incremented via `accounts_types_id_seq`, indexed by `account_type_id_idx`
`name`	`character varying`	No	No unique on name
`can_singin`	`boolean`	No	Default to false, flag used to mark which role can request an account via the login form

Table 6: `account_type` columns

The index on the primary key duplicates the index, this creates noise and has no real impact but it's worth noting. The `name` column has no constraint and thus a new account type named Administrator could be added. As the RBAC logic depends on the name, it would be worth adding, see Section 5.
The column `can_singin` has a typo.

7.2 `users` table

Stores the users account. The `type` column is a foreign key referencing the `account_type.id`. This determines the role of the user and its capabilities within the application, see Section 5.

Column	Type	Nullable	Notes
`id`	`integer`	No	Primary key, auto-incremented via `users_id_seq`.

Column	Type	Nullable	Notes
`username`	`varchar`	No	Must be unique (`users_un`). Indexed by `users_username_idx`.
`password`	`varchar`	Yes	Hashed credential. Nullable to support accounts without passwork (passkey I presume).
`email`	`varchar`	Yes	Contact address, also used by the DEK derivation workflpw.
`totp`	`varchar`	Yes	TOTP secret for two-factor authentication.
`active`	`boolean`	No	Defaults to `true`. Set to `false` to disable login without deleting the row.
`type`	`integer`	No	FK → `account_type.id`. Determines the user's role.

Table 7: `users` columns

Constraints:

- `users_un`, unique constraint on `username`
- `users_fk`, foreign key on `type` referencing `account_type.id`

Indexes:

- `users_username_idx`, B-tree index on `username`

7.3 `webauthn` table

This table stores WebAuthn passkey credentials registered by users. I have not yet seen how and where in the application a user can do it.

Column	Type	Nullable	Notes
`id`	`integer`	No	Auto-incremented via `webauthn_id_seq`.
`user_id`	`integer`	No	No foreign key constraint on this column.
`ukey`	`varchar`	No	Linked to WebAuthn workflow
`credential_id`	`varchar`	No	Linked to WebAuthn workflow
`pub_key`	`varchar`	No	Linked to WebAuthn workflow
`sign_count`	`integer`	No	Linked to WebAuthn workflow
`key_name`	`varchar`	Yes	Readable label for the key
`created_on`	`timestamp`	No	When the row was created
`last_usage`	`timestamp`	Yes	Last time the key was used
`active`	`boolean`	No	Wether the key is active or not. Default to true
`usage_counter`	`integer`	No	Linked to WebAuthn workflow

Table 8: `webauthn` columns

Constraints:
- No explicit foreign key constraint defined for `user_id` referencing `users.id`.
- No explicit primary key constraint defined for `id`. This can hurt performance as the id column is often queried

Indexes:
- No index on `id` column.

7.4 `cf` table

This table must store the consent forms of the donors. It's quite minimal but it stores the consent form file as `varchar` data within the database. As `varchar` expects text formatted with UTF8, an encoding might append server-side before uploading the file. A `blob` field would be better as this is made to store binary data. Or it's actually storing a file path.

Column	Type	Nullable	Notes
`id`	`integer`	No	Auto-incremented via `cf_id_seq`.
`uuid`	`uuid`	No	Probably linked to the uuid of a submission ?
`data`	`varchar`	No	Either encoded data file or a file path
`email`	`varchar`	No	Email not encrypted ?

Table 9: `cf` columns

Constraints:
- No explicit primary key constraint on `id`. This can hurt performance as the id column is often queried
- No explicit foreign key constraint on `uuid` which is probably referencing `submission.uuid`

Indexes:
- No index on `id` column.

7.5 `donor_dek` table

This table stores the DEK for each donor. It will store all the values necessary to generate the DEK again and check it agains the `dek_check` column.

Column	Type	Nullable	Notes
`id`	`integer`	No	Auto-incremented via `donor_dek_id_seq`.
`donor_name`	`varchar`	No	Probably linked to the user's username ?
`salt`	`varchar`	No	Salt used in the DEK generation
`iterations`	`integer`	No	Number of iterations used in the DEK generation algorithm (limited to IntMax)
`algo`	`varchar`	No	Name of the algo used in the DEK generation

Column	Type	Nullable	Notes
`hash`	`varchar`	No	Hashing algorithm used in the DEK generation
`dek`	`varchar`	Yes	The actual DEK generated
`dek_check`	`varchar`	No	The encrypted DEK using AES to regenerate the DEK if needed.

Table 10: `donor_dek` columns

Constraints:

- No explicit primary key constraint on `id`. This can hurt performance as the id column is often queried
- No explicit foreign key constraint defined for `donor_name` referencing `users.username`.

Indexes:

- No index for `id` column.

7.6 `signin_requests` table

This table stores the request for new user accounts for the types with `can_singin` true. See Section 5.

Column	Type	Nullable	Notes
`id`	`integer`	No	Auto-incremented via `signin_requests_id_seq`.
`first_name`	`varchar`	No	User's firstname from the request form
`last_name`	`varchar`	No	User's lastname from the request form
`email`	`varchar`	No	User's email from the request form
`account_type`	`integer`	No	Id of the account type chosen by user in request form
`request_time`	`timestamp`	No	Time at which the request was sent
`uuid`	`uuid`	No	Uuid of the request
`validation_time`	`timestamp`	Yes	The time at which the request was validated
`assertion_response`	`varchar`	Yes	Seems linked to WebAuthn workflow
`status`	`varchar`	No	Status of the request, default to pending
`username_id`	`integer`	No	Id from the sequence of the specific role chosen (e.g. Submitter -> `username_submitter_id` sequence)

Table 11: `signin_requests` columns

Constraints:

- No explicit primary key constraint on `id`. This can hurt performance as the id column is often queried

Indexes:

- No index on `id` column.

7.7 `submissions` table

This table holds the data for creating a new donor. See Section 6.

Column	Type	Nullable	Notes
`id`	`integer`	No	Auto-incremented via `submissions_id_seqo`.
`email_aes`	`varchar`	No	Donor's email encrypted using aes
`email_hash`	`varchar`	No	Donor's email hash using pbkdf2
`nickname`	`varchar`	Yes	Donor's nickname from the creation form
`donor_id`	`integer`	Yes	Id of the donor from the sequence `username_donor_seq`
`status`	`varchar`	No	Status of the submission, default in code to pending
`created_time`	`timestamp`	No	Time at which the submission is created
`update_time`	`timestamp`	No	Time at which the submission was last updated
`submitter_id`	`integer`	No	Id of the submitter
`uuid`	`uuid`	No	Uuid of the donor according to the code
`consent_form`	`boolean`	No	Whether the consent form has been uploaded or not. Default to false

Table 12: `submissions` columns

Constraints:

- No explicit primary key constraint on `id`. This can hurt performance as the id column is often queried
- No explicit foreign key constraint on `submitter_id`. This makes the schema less readable.

Indexes:

- No index on `id` column.

7.8 `files_type` table

Stores the different files type available in the application.

Column	Type	Nullable	Notes
`id`	`integer`	No	Auto-incremented via `files_type_id_seq`.

Column	Type	Nullable	Notes
`name`	`varchar`	No	Name of the type
`desc`	`varchar`	Yes	The string to represent the type

Table 13: `files_type` columns

Constraints:
- No explicit primary key constraint on `id`. This can hurt performance as the id column is often queried

Indexes:
- No index on `id` column.

The default inserts are the following:
- Consent form
- Tenprint card front
- Tenprint card back
- Mark target
- Mark incidental
- TP NIST file

7.9 `files` table

This table stores the files data and its metadata like height, resolution, format, etc…

Column	Type	Nullable	Notes
`id`	`integer`	No	Auto-incremented via `files_id_seq`.
`creator`	`integer`	Yes	Linked to the user creating the file, no foreign key
`creation_time`	`timestamp`	No	Time at which the file is created
`folder`	`integer`	Yes	Organisation column, there's a `exercises_folder` table, maybe should be a foreign key
`filename`	`varchar`	No	Name of the file
`type`	`integer`	Yes	Type of file, no foreign key
`size`	`bigint`	Yes	Size of the file
`uuid`	`uuid`	Yes	Uuid of the file
`data`	`varchar`	Yes	Data of the file in UTF8
`width`	`integer`	Yes	Width of the file
`height`	`integer`	Yes	Height of the file
`format`	`varchar`	Yes	Format of the file

Column	Type	Nullable	Notes
`resolution`	`integer`	Yes	Resolution of the file
`note`	`varchar`	Yes	Note about the file
`quality`	`integer`	Yes	Quality of the file, no foreign key

Table 14: `files` columns

Constraints:

- No explicit primary key constraint on `id`. This can hurt performance as the id column is often queried
- No explicit foreign key constraint on `creator` to `users.id`.
- No explicit foreign key constraint on `folder` to, I suspect, `exercises_folder.id`
- No explicit foreign key constraint on `quality` to, I suspect, `quality_type.id`

Indexes:

- No index on `id` column.

This looks like a very broad table encompassing images and pdf alike.

7.10 `files_v` view

This view queries all the columns from the `files` table without the data column.

7.11 `segments_locations` table

Stores the location and orientation of individual finger segments from a tenprint card image.

Column	Type	Nullable	Notes
`id`	`integer`	No	Auto-incremented via `segments_locations_id_seq`.
`tenprint_id`	`uuid`	No	Uuid of the tenprint card, no foreign key
`fpc`	`integer`	No	Finger position code (e.q. right thumb = 1), no foreign key
`x`	`numeric`	No	x coordinate of the bounding box
`y`	`numeric`	No	y coordinate of the bounding box
`width`	`numeric`	No	Width of the bounding box
`height`	`numeric`	No	Height of the bounding box
`orientation`	`integer`	No	Orientation of the bounding box

Table 15: `segments_locations` columns

Constraints:

- No explicit primary key constraint on `id`. This can hurt performance as the id column is often queried

- No explicit foreign key constraint on `tenprint_id` to another table ?
- No explicit foreign key constraint on `fpc` to `pc.id`

Indexes:

- No index on `id` column.

7.12 `files_segments` table

This seems to store the actual extracted finger image data. It is used to serve the image.

Column	Type	Nullable	Notes
`id`	`integer`	No	Auto-incremented via `files_segment_id_seq`.
`tenprint`	`uuid`	No	Uuid of the tenprint card, no foreign key
`pc`	`integer`	No	Position code (e.q. right thumb = 1), no foreign key
`data`	`varchar`	No	Data encoded UTF8
`uuid`	`uuid`	No	Uuid of the row

Table 16: `files_segments` columns

Constraints:

- No explicit primary key constraint on `id`. This can hurt performance as the id column is often queried
- No explicit foreign key constraint on `tenprint` to another table ?
- No explicit foreign key constraint on `pc` to `pc.id`

Indexes:

- No index on `id` column.

7.13 `files_segements_v` view

This view queries all the columns of the `files_segments` table expect the data column.

7.14 `thumbnails` table

This table is used to store thumbnails of the different images. Are the thumbnails also encrypted ?

Column	Type	Nullable	Notes
`id`	`integer`	No	Auto-incremented via `thumbnails_id_seq`.
`uuid`	`uuid`	No	Uuid of the row
`width`	`integer`	No	Width of the thumbnail
`height`	`integer`	No	Height of the thumbnail
`size`	`integer`	No	Size of the thumbnail

Column	Type	Nullable	Notes
`data`	`varchar`	No	Data encoded UTF8
`format`	`varchar`	No	Format of the thumbnail

Table 17: `thumbnails` columns

Constraints:
- No explicit primary key constraint on `id`. This can hurt performance as the id column is often queried

Indexes:
- No index on `id` column.

7.15 `quality_type` table

Stores the different quality values possible in the web applciation.

Column	Type	Nullable	Notes
`id`	`integer`	No	Auto-incremented via `newtable_id_seq`.
`name`	`varchar`	No	Name of the quality type

Table 18: `quality_type` columns

Constraints:
- No explicit primary key constraint on `id`. This can hurt performance as the id column is often queried
- No unique constraint on the `name` column, which would be interesting for enforcing business logic

Indexes:
- No index on `id` column.

Lists of quality type values:
- Prestine (typo for Pristine)
- Good
- Bad

7.16 `tenprint_zones_location` table

Stores the possible zone location for the tenprint ? However, I don't see it used within the application. It's never called in a sql query.

Column	Type	Nullable	Notes
`pc`	`integer`	No	Position code that also acts as a primary key, no foreign key
`side`	`varchar`	No	The side of the zone (either front or back)

Table 19: `tenprint_zones_location` columns

Constraints:
- ▸ No explicit foreign key constraint on `pc` to `pc.id`

Indexes:
- None

Values inserted by default:
- 1, front
- 2, front
- 3, front
- 5, front
- 6, front
- 7, front
- 8, front
- 9, front
- 10, front
- 11, front
- 12, front
- 13, front
- 14, front
- 22, back
- 24, back
- 25, back
- 27, back

7.17 `tenprint_zones` table

This table seems to be a template that would define the expected zones on a tenprint card. However, I don't see it used within the application. It's never called in a sql query.

Column	Type	Nullable	Notes
`id`	`integer`	No	Auto-incremented with `tenprint_templates_id_seq`
`pc`	`integer`	No	Position code, no foreign key
`angle`	`numeric(10, 0)`	Yes	Expected rotation/orientation of the finger in that zone ?
`card`	`integer`	No	References a card type, I don't see any table that could be it ?
`tl_x`	`numeric`	Yes	Top left x coordinate
`tl_y`	`numeric`	Yes	Top left y coordinate
`br_x`	`numeric`	Yes	Bottom right x coordinate

Column	Type	Nullable	Notes
br_y	numeric	Yes	Bottom right y coordinate

Table 20: tenprint_zones columns

Constraints:
- No explicit primary key constraint on id. This can hurt performance as the id column is often queried
- No explicit foreign key constraint on pc to pc.id

Indexes:
- No index on id column.

7.18 mark_info table

This table stores the information regarding a mark.

Column	Type	Nullable	Notes
id	integer	No	Auto-incremented with mark_info_id_seq
uuid	uuid	No	Uuid for the specific row
pfsp	varchar	Yes	Police or private forensics science providers ?
detection_technic	varchar	No	Detection technique used to retrieve the mark (e.q. Powder dusting), no foreign key
surface	varchar	Yes	Surface on which the mark was retrieved

Table 21: mark_info columns

Constraints:
- No explicit primary key constraint on id. This can hurt performance as the id column is often queried
- No explicit foreign key constraint on detection_technic to detection_technics.id as it's a varchar. This seems strange to me.
- No explicit foreign key constraint on surface to surfaces.id as it's a varchar. This seems strange to me.

Indexes:
- No index on id column.

7.19 gp table

Stores the General Pattern found in fingerprints.

Column	Type	Nullable	Notes
id	integer	No	Auto-incremented with gp_id_seq

Column	Type	Nullable	Notes
`name`	`varchar`	No	Name of the General Pattern
`div_name`	`varchar`	No	The name of the div corresponding ?

Table 22: `gp` columns

Constraints:
- No explicit primary key constraint on `id`. This can hurt performance as the id column is often queried

Indexes:
- No index on `id` column.

Default inserted values:
- 1, unknown, unknown
- 2, left loop, ll
- 3, right loop, rl
- 4, whorl, whorl
- 5, arch, arch
- 6, central pocket loop, cpl
- 7, double loop, dl
- 8, missing/amputated, ma
- 9, scarred/mutilated, sm

7.20 `donor_fingers_gp` table

Many-to-Many relationship table for General pattern, Finger Position Code and Users.

Column	Type	Nullable	Notes
`id`	`integer`	No	Auto-incremented with `donor_fingers_gp_id_seq`
`donor_id`	`integer`	No	Id of the donor, no foreign key
`fpc`	`integer`	No	Id of the finger position code, no foreign key
`gp`	`integer`	No	Id of the general pattern, no foreign key

Table 23: `gp` columns

Constraints:
- No explicit primary key constraint on `id`. This can hurt performance as the id column is often queried
- No explicit foreign key constraint on `donor_id` to `users.id`
- No explicit foreign key constraint on `fpc` to `pc.id`
- No explicit foreign key constraint on `gp` to `gp.id`

Indexes:
- No index on `id` column.

7.21 `pc` table

Stores the values available for the Position Code in the application.

Column	Type	Nullable	Notes
`id`	`integer`	No	Auto-incremented with `fpc_id_seq`
`name`	`varchar`	No	Name of the finger position code

Table 24: `pc` columns

Constraints:

- No explicit primary key constraint on `id`. This can hurt performance as the id column is often queried
- No explicit unique constraint on `name`, would be necessary to not have two pc with the same name.

Indexes:

- No index on `id` column.

Default inserted values:

- 1, Right thumb
- 2, Right index
- 3, Right middle
- 4, Right ring
- 5, Right little
- 6, Left thumb
- 7, Left index
- 8, Left middle
- 9, Left ring
- 10, Left little
- 11, Right thumb slap
- 12, Left thumb slap
- 13, Right control slap
- 14, Left control slap
- 22, Right writer palm
- 24, Left writer palm
- 25, Right lower palm
- 27, Left lower palm
- 1000, All rolled

7.22 `detection_technics` table

Stores the available values for the detection technique in the application.

Column	Type	Nullable	Notes
`id`	`integer`	No	Auto-incremented with `detection_technics_id_seq`
`name`	`varchar`	No	Name of the detection technique

Table 25: `detection_technics` columns

Constraints:
- No explicit primary key constraint on `id`. This can hurt performance as the id column is often queried
- No explicit unique constraint on `name`, would be necessary to not have two detection techniques with the same name.

Indexes:
- No index on `id` column.

Default inserted values:
- There are 49 default inserted values I only put the 5 first ones
- 1, Black Powder
- 2, Ninhydrine
- 3, 1,2-Indanedione
- 4, Cyanoacrylate fuming (CA)
- 5, Black Powder Suspension (BPS)
- 6, Optical

7.23 `surfaces` table

Stores the available values for the surfaces in the application.

Column	Type	Nullable	Notes
`id`	`integer`	No	Auto-incremented with `surfaces_id_seq`
`name`	`varchar`	No	Name of the surface

Table 26: `surfaces` columns

Constraints:
- No explicit primary key constraint on `id`. This can hurt performance as the id column is often queried
- No explicit unique constraint on `name`, would be necessary to not have two detection techniques with the same name.

Indexes:
- No index on `id` column.

Default inserted values:
- 1, Paper
- 2, Plastic
- 3, Glass

- 4, Bag
- 5, Bottle
- 6, Sticky side
- 7, Non-adhesive side
- 8, Tape
- 9, Fabric / Cloth
- 10, Metal
- 11, Cartridge
- 12, Wood
- 13, Porcelain

7.24 `exercises` table

Stores the exercises information.

Column	Type	Nullable	Notes
`id`	`integer`	No	Auto-incremented via `exercises_id_seq`.
`uuid`	`uuid`	No	Uuid of the exercise.
`trainer_id`	`integer`	No	Id of the trainer who created the exercise, no foreign key.
`creationtime`	`timestamp with time zone`	No	Time at which the exercise was created.
`name`	`character varying`	No	Name of the exercise.
`active`	`boolean`	No	Whether the exercise is active. Defaults to `true`.

Table 27: `exercises` columns

Constraints:
- No explicit primary key constraint on `id`. This can hurt performance as the id column is often queried.
- No explicit foreign key constraint on `trainer_id` to `users.id`.

Indexes:
- No index on `id` column.

7.25 `exercises_folder` table

This table is a many-to-many relationship table linking marks to exercise folders.

Column	Type	Nullable	Notes
`id`	`integer`	No	Auto-incremented via `exercises_folder_id_seq`.
`mark`	`uuid`	No	Uuid of the mark, references `mark_info.uuid`, no foreign key.
`folder`	`uuid`	No	Uuid of the folder

Table 28: `exercises_folder` columns

Constraints:
- No explicit primary key constraint on `id`. This can hurt performance as the id column is often queried.
- No explicit foreign key constraint on `mark` to `mark_info.uuid`.

Indexes:
- No index on `id` column.

7.26 `exercises_trainee_list` table

This table is a many-to-many relationship table linking trainees to exercise folders.

Column	Type	Nullable	Notes
`id`	`integer`	No	Auto-incremented via `exercises_trainee_list_id_seq`.
`user_id`	`integer`	No	Id of the trainee, references `users.id`, no foreign key.
`folder`	`uuid`	No	Uuid of the folder, no foreign key

Table 29: `exercises_trainee_list` columns

Constraints:
- No explicit primary key constraint on `id`. This can hurt performance as the id column is often queried.
- No explicit foreign key constraint on `user_id` to `users.id`.
- No explicit foreign key constraint on `folder` to `exercises_folder.folder`.

Indexes:
- No index on `id` column.

7.27 `activities` table

Stores the available activity values in the application.

Column	Type	Nullable	Notes
`id`	`integer`	No	Auto-incremented via `activities_id_seq`.
`name`	`character varying`	No	Name of the activity.

Table 30: `activities` columns

Constraints:

- No explicit primary key constraint on `id`. This can hurt performance as the id column is often queried.
- No explicit unique constraint on `name`, would be necessary to not have two activities with the same name.

Indexes:
- No index on `id` column.

There's no default values in this table.

7.28 `distortion` table

Stores the available distortion values in the application.

Column	Type	Nullable	Notes
`id`	`integer`	No	Auto-incremented via `distortion_id_seq`.
`name`	`character varying`	No	Name of the distortion type.

Table 31: `distortion` columns

Constraints:
- No explicit primary key constraint on `id`. This can hurt performance as the id column is often queried.
- No explicit unique constraint on `name`, would be necessary to not have two distortion types with the same name.

Indexes:
- No index on `id` column.

Default inserted values:
- 1, drag
- 2, twist
- 3, unknown
- 4, none

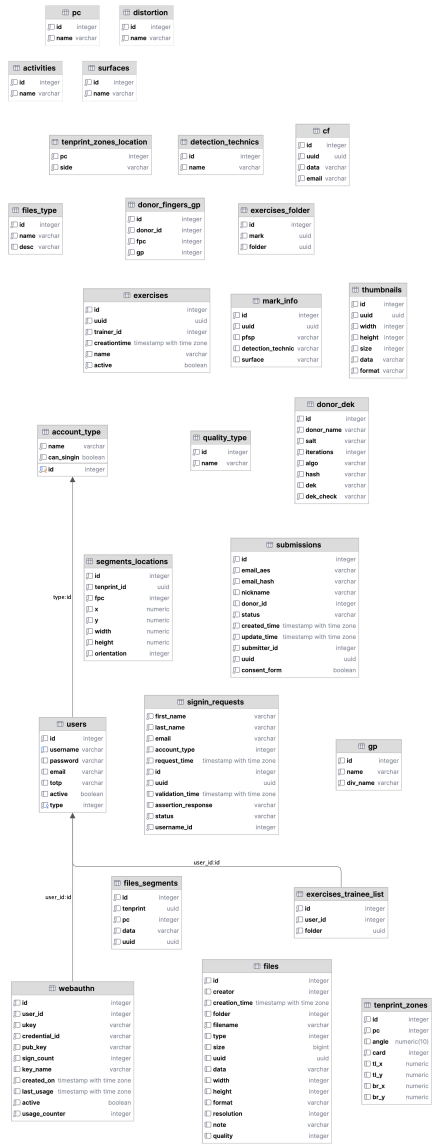

Figure 4: Auto-generated schema of the database tables

8 Architecture

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

9 Implementation

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

10 Results

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae.

11 Conclusion

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et.

Bibliography

Figures table

Figure 1 Inital planning for the project 14
Figure 2 Repositories dependencies 20
Figure 3 DEK Generation Schema 31
Figure 4 Auto-generated schema of the database tables 49

Tabs list

Table 1 Account types table ... 22
Table 2 Routes of interest for Donor role ... 24
Table 3 Routes of interest for Submitters role ... 25
Table 4 Routes of interest for Trainer role ... 25
Table 5 Routes of interest for AFIS role ... 26
Table 6 `account_type` columns ... 32
Table 7 `users` columns ... 32
Table 8 `webauthn` columns ... 33
Table 9 `cf` columns ... 34
Table 10 `donor_dek` columns ... 34
Table 11 `signin_requests` columns ... 35
Table 12 `submissions` columns ... 36
Table 13 `files_type` columns ... 36
Table 14 `files` columns ... 37
Table 15 `segments_locations` columns ... 38
Table 16 `files_segments` columns ... 39
Table 17 `thumbnails` columns ... 39
Table 18 `quality_type` columns ... 40
Table 19 `tenprint_zones_location` columns ... 40
Table 20 `tenprint_zones` columns ... 41
Table 21 `mark_info` columns ... 42
Table 22 `gp` columns ... 42
Table 23 `gp` columns ... 43
Table 24 `pc` columns ... 44
Table 25 `detection_technics` columns ... 45
Table 26 `surfaces` columns ... 45
Table 27 `exercises` columns ... 46
Table 28 `exercises_folder` columns ... 47
Table 29 `exercises_trainee_list` columns ... 47
Table 30 `activities` columns ... 47

Table 31 `distortion` columns 48
Table 32 Journal de travail 61

Appendices

I Tools used

Lorem ipsum dolor sit amet, consectetur adipiscing elit, sed do eiusmod tempor incididunt ut labore et dolore magnam aliquam quaerat voluptatem. Ut enim aeque doleamus animo, cum corpore dolemus, fieri tamen permagna accessio potest, si aliquod aeternum et infinitum impendere malum nobis opinemur. Quod idem licet transferre in voluptatem, ut postea variari voluptas distinguique possit, augeri amplificarique non possit. At etiam Athenis, ut e patre audiebam facete et urbane Stoicos irridente, statua est in quo a nobis philosophia defensa et collaudata est, cum id, quod maxime placeat, facere possimus, omnis voluptas assumenda est, omnis dolor repellendus. Temporibus autem quibusdam et aut officiis debitis aut rerum necessitatibus saepe eveniet, ut et voluptates repudiandae sint et molestiae non recusandae. Itaque earum rerum defuturum, quas natura non depravata desiderat. Et quem ad me accedis, saluto: 'chaere,' inquam, 'Tite!' lictores, turma omnis chorusque: 'chaere, Tite!' hinc hostis mi Albucius, hinc inimicus. Sed iure Mucius. Ego autem mirari satis non queo unde hoc sit tam insolens domesticarum rerum fastidium. Non est omnino hic docendi locus; sed ita prorsus existimo, neque eum Torquatum, qui hoc primus cognomen invenerit, aut torquem illum hosti detraxisse, ut aliquam ex eo est consecutus? – Laudem et caritatem, quae sunt vitae sine metu degendae praesidia firmissima. – Filium morte multavit. – Si sine causa, nollem me ab eo delectari, quod ista Platonis, Aristoteli, Theophrasti orationis ornamenta neglexerit. Nam illud quidem physici, credere aliquid esse minimum, quod profecto numquam putavisset, si a Polyaeno, familiari suo, geometrica discere maluisset quam illum etiam ipsum dedocere. Sol Democrito magnus videtur, quippe homini erudito in geometriaque perfecto, huic pedalis fortasse; tantum enim esse omnino in nostris poetis aut inertissimae segnitiae est aut fastidii delicatissimi. Mihi quidem videtur, inermis ac nudus est. Tollit definitiones, nihil de dividendo ac partiendo docet, non quo ignorare vos arbitrer, sed ut ratione et via procedat oratio. Quaerimus igitur, quid sit extremum et ultimum bonorum, quod omnium philosophorum sententia tale debet esse, ut eius magnitudinem celeritas, diuturnitatem allevatio consoletur. Ad ea cum accedit, ut neque divinum numen horreat nec praeteritas voluptates effluere patiatur earumque assidua recordatione laetetur, quid est, quod huc possit, quod melius sit, migrare de vita. His rebus instructus semper est in voluptate esse aut in armatum hostem impetum fecisse aut in poetis evolvendis, ut ego et Triarius te hortatore facimus, consumeret, in quibus hoc primum est in quo admirer, cur in gravissimis rebus non delectet eos sermo patrius, cum.

II Work diary

Date	Description	Rech. [h]	Dev. [h]	Rapport [h]	Admin [h]
					Le journal de travail continue à la page suivante.
05.03.2026	Kick-off meeting avec le mandant	0	0	0	2
12.03.2026	Recherche sur l'état de l'art	4	0	0	0
19.03.2026	Mise en place de l'environnement de développement et documentation	0	7	1	0
25.03.2026	Rédaction de l'état de l'art	0	0	4	0
25.03.2026	Dev de la partie interface	0	0	4	0
xx.xx.2026	Test	0	0	4	0
xx.xx.2026	Test	0	0	4	0

Date	Description	Rech. [h]	Dev. [h]	Rapport [h]	Admin [h]
xx.xx.2026	Test	0	0	4	0
xx.xx.2026	Test	0	0	4	0
xx.xx.2026	Test	0	0	4	0
xx.xx.2026	Test	0	0	4	0
xx.xx.2026	Test	0	0	4	0
xx.xx.2026	Test	0	0	4	0
xx.xx.2026	Test	0	0	4	0
xx.xx.2026	Test	0	0	4	0

Table 32: Journal de travail